Feature guides

Use these recipes after the core business-and-currency loop works. Each section names the assets, manager, and runtime call that complete the feature.

Table of contents

Automate businesses with managers

1. Add `ManagersManager` to the scene manager GameObject.
2. Create **EasyIdleGame > Managers > Manager** below a Resources folder.
3. Configure cost, locks, metadata, and optional manager groups.
4. Assign the manager to one or more `Business.manager` fields.
5. Set `autoProduceOnLevel` and `autoCollectOnLevel`.
6. Add passive `UpgradeBlock` entries for always-on effects after the manager is unlocked.
7. Configure the active upgrade, active duration, and cooldown when the manager has a triggered ability.
8. Buy/upgrade/activate the holder through `ManagersManager` or `ManagerDisplayer`.

Manager multipliers can affect production, speed, purchase cost, XP, duration, cooldown, and other upgrade types supported by the resolver. Contextual blocks can target a specific manager.

The default passive list contains a speed multiplier. Remove or replace it if a new manager should provide automation without a speed bonus.

Active manager abilities and offline simulation

Active abilities begin through `ManagersManager.Instance.TryActivateManager(manager)`; a manager at level zero cannot activate. Render the active duration and the cooldown as separate states and refresh them from manager events.

Offline calculation can simulate active manager behavior, so a custom manager effect must define whether and how it applies in offline mode. Use `ManagersManager.iBuyable` for generic store flows and the normal manager APIs for level, activation, and assignment changes.

Create permanent upgrades

1. Add `UpgradesManager`.
2. Create **EasyIdleGame > Upgrades > Upgrade** below Resources.
3. Add cost, maximum buyable amount, locks, and metadata.
4. Add one or more `UpgradeBlock` entries.
5. Leave the block filters empty for a global effect, or set `targetBusinessGroups` on the upgrade for normal business-group targeting.

6. Present the asset with `UpgradeDisplayer` or call the buyable manager API.

Standard `UpgradeType` values are:

Type	Affects
`production`	Generated output and reward-style amounts
`speed`	Production and merge speed-style effects
`purchaseCost`	Costs for businesses, managers, and boosts
`playerXp`	Player XP rewards
`businessXp`	Business XP rewards
`duration`	Boost or manager-active duration
`dropChance`	Independent output chance
`dropWeight`	Weighted-random relative weight
`productionOutputScaling`	Runtime output-scaling modifiers
`cooldown`	Manager cooldown and merge cooldown
`productionInputCost`	Inputs paid to start production
`upgradePurchaseCost`	Cost of buying Upgrade assets
`levelUpCost`	Cost of upgrading an `IUpgradableHolder`
`mergeInputCost`	Inputs consumed by merge recipes

Most global values multiply. For reductions, use a value below `1`, such as `0.8` for a 20% cost reduction.

Projects can define stable custom `UpgradeType` and `OverrideUpgradeType` identifiers without modifying package enums. See Extend upgrade enums with stable values (`05-scripting-reference.pdf` > `Extend upgrade enums with stable values`) for the enum-overloading pattern, numeric registry rules, Inspector limitations, and project-owned examples.

Target contextual upgrade effects

Use `UpgradeBlock.filters` when an effect should depend on what is happening.

Available filters include:

- Source business or source business groups.
- `SourceType`, such as `Production`, `Purchase`, `RecurringReward`, or `Reset`.
- Target currency or currency groups.
- Target business or business groups.
- Target upgrade or upgrade groups.
- Target boost or boost groups.
- Target manager.
- Target merge recipe.
- Minimum and maximum active business upgrade level.
- Runtime flags: Active, Offline, and Manual.

Filters assigned in one block are cumulative: every configured category must match. Within a typed-group list, any overlapping group matches.

Runtime flags are a mask. A manual active action has both Active and Manual context. Leave the mask empty for no runtime restriction; select Manual alone for manual-only effects.

`UpgradeBlock.operation` controls contextual accumulation. Use Multiply for normal scaling or Add where the resolver supports a flat adjustment, such as drop chance.

Example: to double Coins from offline production by Farm businesses only:

1. Set type to `production` and value to `2`.
2. Set source to `Production`.
3. Add the Farm group to `sourceBusinessGroups`.
4. Set target currency to Coin.
5. Set runtime filter to Offline.

Resolve the final contextual value

Custom preview or scripting calculations must build an `UpgradeEffectContext` for the exact operation and resolve it through `UpgradeEffectResolver`; a global multiplier list omits contextual effects and produces wrong previews. Null or empty filters are broad, so treat every new contextual asset as a targeting review — an unfiltered block applies everywhere its type is read.

Create override upgrades

Override blocks set absolute values. The supported types are:

- `maxCurrencyAmount`
- `idleProductionTimeCap`
- `maxBusinessAmount`

Configure `targetCurrency` or `targetBusiness` where the value should be scoped. When several owned upgrades provide the same override, the highest modified value wins.

`OverrideUpgradeModifierBlock` improves existing override sources but does not create one. It can add or multiply, and can target a specific source upgrade. Resolution is:

`(source override + additive modifiers) × multiplicative modifiers`

Use this separation for “wallet size” and “wallet efficiency” upgrades.

Queue timed upgrades

1. Set `Upgrade.upgradeDurationInSeconds` above zero.
2. Choose `UpgradesManager.queueMode`:
   - `Parallel`: each purchase completes after its own duration.
   - `Sequential`: each pending purchase completes after the previous queued end time; bulk amounts extend the duration proportionally.
3. Show pending state from `UpgradeHolder.pendingUpgrades`.
4. Listen to `OnUpgradeApplied` to redraw completed effects.
5. Call `SkipUpgradeWait(upgrade)` when a skip action should immediately apply every pending entry for that asset.

Pending entries are part of the serialized `UpgradeHolder` and are restored with save data.

Create and activate boosts

Add `BoostsManager`, then create one of the built-in boost assets:

Boost	Behavior
`GenericMultiplierBoost`	Applies timed `UpgradeBlock` effects
`AutoProduceBoost`	Temporarily automates matching businesses
`TimeSkipBoost`	Immediately calculates and applies a block of offline-style production

For a multiplier boost:

1. Configure cost, locks, metadata, and boost groups.
2. Add upgrade blocks.
3. Set a `Duration`.
4. Add typed target business groups, or leave them empty for global targeting.
5. Enable `mergeRepeatedUses` when matching activations should extend the existing timer.
6. Buy the boost into inventory and activate it through `BoostsManager` or `BoostDisplayer`.

Matching active boosts merge only when their relevant type, groups, and effects match. Otherwise, they remain separate active entries.

Boost inventory and active state are separate: buying or granting a boost adds inventory, and using it creates or updates an active holder. Subscribe to `OnActiveBoostsChanged` for UI; inventory amount does not imply an active effect.

Runtime-created boosts

For a runtime-calculated effect, clone the boost ScriptableObject before changing filters, targets, multipliers, or duration; mutating the shared asset leaks state to every other consumer. Activate or refresh the clone through `BoostsManager.SetActiveBoost`, then call `RemoveActiveBoost` and destroy the clone during teardown so stale holders do not accumulate. When merging is disabled, manage replacement or removal explicitly.

Add a time-skip item

1. Create **EasyIdleGame > Boosts > Time Skip Boost**.
2. Set `skipTime`.
3. Enable `simulateBoosts` if active boost timers should advance during the skipped time.
4. Grant or sell the boost.
5. Activate it through `BoostsManager`.

Activation calls `ProfitCalculator.CalculateProfit`, applies the returned `ProfitData`, invokes `BoostsManager.OnTimeSkip`, and returns no active timer holder.

Do not separately apply the same skipped duration, or the player receives duplicate progress.

Create locations and travel

1. Add `LocationsManager`.
2. Create **EasyIdleGame > Locations > Location** assets below Resources.
3. Configure metadata, sort order, optional location groups, locks, and one-time purchase cost.
4. Add starter reward tables if the first unlock should grant resources or businesses.
5. Assign `defaultActiveLocation` if gameplay must begin in a world.
6. Call:

```
bool traveled = LocationsManager.Instance.TryTravelTo(location);
```

If the location is not owned, `TryTravelTo` first attempts to unlock it, pays the cost, applies starter outputs once, and then changes the active location. `OnLocationChanged` is the appropriate hook for world UI, music, and scene presentation.

`Location` can also be used in locks and attached to a business through `Business.location`.

Subclass Location for project world data

Subclass `Location` when a world needs additional authored data, and keep economic travel in `LocationsManager` instead of building a parallel location system. Typed queries — `GetAllLocations<T>()`, `GetUnlockedLocations<T>()`, and `TryGetActiveLocation<T>(out location)` — return the subclass, so world UI can load, sort, and render project data from the same assets. Run scene or animation transitions before or around `TryTravelTo`, then react to `OnLocationChanged`; the manager owns economic active-location state, not visual transitions.

Starter output tables belong to unlock and reset restoration; do not grant them during ordinary travel. Restoring starters after prestige is covered in Preserve location starter state (`04-persistence-offline-prestige.pdf`).

Add player progression and achievements

Player progression

1. Add `PlayerStats` .
2. Configure the player `Level` data and level rewards.
3. Set each business's `xpsAddPerPurchaseToPlayer` when business acquisition should grant player XP.
4. Create `CustomStat` assets for project-specific counters.
5. Change custom stats through `PlayerStats.IncrementStat(stat, amount)` so persistence, locks, events, and save-worthy notifications stay synchronized.

Player stats also track lifetime currency, businesses obtained, and boosts obtained. Lifetime tracking is distinct from current holder amounts.

Achievements

1. Add `AchievementsManager` .
2. Create **EasyIdleGame > Player Stats > Achievement** below Resources.
3. Configure locks, reward tables, optional achievement groups, and tier scaling.
4. Present it with `AchievementDisplayer` .
5. Claim through the manager and use the returned/applied outputs for feedback.

Use `TotalAmount` locks for lifetime goals such as “earn 1,000 Coins.” Use `CurrentAmount` only when the player must retain the amount at claim time.

An `AchievementHolder` tracks the current claim tier, the claimed amount, and the terminal state. Locks are evaluated for the current tier and scale by `claimMultiplier` for up to `claimAmount` claims; display the scaled current-tier requirement from `GetCurrentTierLocks()` , not only the base authored value. With `autoClaim` enabled, eligible rewards are applied during the manager update. Manual UI calls `TryClaimAchievement(achievement, out outputs)` and uses the returned outputs for feedback. An achievement with no locks is immediately eligible, and one with no rewards still advances its claim tier.

Build a daily reward calendar

1. Add `DailyRewardsManager` .
2. Create reusable `Reward` assets below Resources.
3. Add exact-day `DailyRewardHolder` entries to `dailyRewards` .
4. Leave `combineExactDayRewards` enabled to combine every exact-day entry that matches; disable it only to preserve the legacy first-match behavior.
5. Add repeating `LevelRewardHolder` entries to `eachDayReward` ; use a positive level such as 7 to grant whenever the day is divisible by 7. Zero is not a valid divisor.
6. Enable `scaleEachDayReward` only when repeating output should multiply by the current day number.
7. Set `dayLengthHours` ; use 24 in production and a shorter value only for testing/events.
8. Display the calendar with `DailyRewardListDisplayer` or call `GetCalendarView` .
9. Claim with `TryClaimDailyRewards(day, out outputs)` .

`GetMissedDays` reports unclaimed reward-bearing days after the last claimed day. `initDate` and `claimedDays` are saved by `SaveFile` .

`TryClaimDailyRewards` validates the requested day itself: it returns `false` for already-claimed, non-positive, future, and rewardless days, and only successful claims are recorded in `claimedDays` . By default, all matching exact-day and repeating entries are combined. Set `combineExactDayRewards` to `false` only when compatibility with the legacy first-match behavior is required.

Simulation and clock policy

Render the calendar from `GetCalendarView` and `GetMissedDays` preview data, but grant only through `TryClaimDailyRewards` and show its returned outputs. `SimulateNextDay` is a testing/demo tool, not a production clock policy. The package uses local device time; missed-day presentation, clock-rollback handling, anti-tamper checks, and server-authoritative time are game-design decisions that belong in project code.

Connect the shop to currency, ads, and IAP

1. Add `ShopManager`.
2. Create **EasyIdleGame > Shop > Shop Item** below Resources.
3. Configure exactly one primary payment method:
    - In-game `Input` cost.
    - `buyWithAd`.
    - Real-money `productId`.
4. Add reward tables, locks, groups, and `maxPurchases` (`0` means infinite).
5. Call `ShopManager.AttemptPurchase(item)`.

Payment priority for a misconfigured multi-cost item is Ad, then Real Money, then In-Game Cost. The inspector warns about this; configure one method instead of depending on priority.

For an ad or IAP:

1. Subscribe to `OnAttemptAdPurchase` or `OnAttemptRealMoneyPurchase`.
2. Open the platform flow.
3. After verified success, call `CompleteExternalPurchase(item, sourceType)` exactly once.

`CompleteExternalPurchase` increases the holder and immediately grants the reward tables. The normal in-game `AttemptPurchase` path eventually uses the same completion method, so it also grants rewards immediately and leaves holder amount behind.

`TryConsumeItem` removes holder amount and grants the reward tables again. Do not consume a normally or externally completed item unless a second reward grant is intentional. The built-in flow does not provide an inventory-only purchase mode; implement one in project code if rewards must wait until consumption. Bulk external completions floor the requested amount, reject non-positive values, and clamp the grant to the remaining purchase limit; consumption rejects non-positive amounts.

External purchase completion contract

The package delegates the store and ad work: it does not validate receipts, restore purchases, implement consent, or decide ad success. Complete only after the platform callback is verified, and never on button press. Keep one pending request per integration, or attach a project transaction identifier, so a late callback cannot complete the wrong item. Do not substitute `ForceBuyItemsForFree` for verified external completion; it bypasses the external-completion semantics and records the wrong `SourceType`.

Schedule limited-time offers

1. Add both `ShopManager` and `OffersManager`.
2. Add `OfferPoolItem` entries with an exact shop item or a shop item group.
3. Set weight, active duration, and the minimum/maximum wait between offers.
4. Subscribe to `OnOfferBecameAvailable` and `OnOfferExpired`.
5. Call the shop purchase flow for the active offer.
6. Call `TryConsumeCurrentOffer` when purchase or dismissal should end it early.

Locked and maxed shop items are skipped. Group entries expand to matching shop holder items. `TriggerOffer` is available as an editor/runtime test action.

Group expansion only examines holders already created by `ShopManager`. Call `GetOrAddHolder` for group-backed items before offers begin, or add exact item entries. `TryConsumeCurrentOffer` ends the offer without invoking `OnOfferExpired`.

Purchase first, consume after confirmation

Consuming an offer is not purchasing it. Route the purchase through `ShopManager` first, then call `TryConsumeCurrentOffer` after the operation completes — for ad or IAP offers that means after the verified external completion, not on button press. If the pool has no eligible positive-weight item, hide the offer surface and schedule another attempt instead of showing an empty popup.

Add production modifier assets

Use a `ProductionOutputModifierAsset` when production output needs project-specific scaling beyond standard upgrades.

All registered modifier assets derive from `ProductionModifierAsset`. Output-stage assets implement `IProductionOutputModifier`; `ProductionOutputModifierAsset` also supplies ordered execution through `IOrderedProductionOutputModifier`.

The included `ProductionBusinessOutputScalingModifierAsset` can:

- Target a producing business.
- Add or replace one scaled output granting any target — currency, business, boost, or manager (`outputCurrency`, `outputBusiness`, `outputBoost`, `outputManager`); every assigned target receives the scaled amount.
- Remove each configured target from the normal production outputs first when `replaceExistingMatchingOutput` is enabled; unconfigured targets stay unchanged.
- Affect active, manual, preview, and offline calculation modes through the shared production context.

Register modifiers through `ProductionModifierManager` or `ProductionModifierRegistry`. Production modifier assets are applied to business production output; they do not globally rewrite unrelated drop-table generation.

Upgrade type `productionOutputScaling` can improve matching scaling modifiers.

Choose context, mode, and order

A modifier receives the calculated output list plus a `ProductionOutputContext` and must return adjusted outputs without mutating shared output assets. Filter on the context and `ProductionOutputCalculationMode` when an effect must not apply to previews, per-second rates, offline simulation, or manual production. Order matters when several modifiers change the same output; `ProductionOutputModifierAsset` supplies ordered execution.

Register and unregister runtime modifiers with the owning component's lifetime; a stale static registration affects later scenes and tests. Verify live and offline results together: open `EasyIdleGame > Demos > FeatureDemos > ProductionModifiers > ProductionModifiers.unity`, which compares base output with modified output across actual, preview, per-second, and offline calculation modes.

UI setup — displayers, uGUI, UI Toolkit presenters, and event-driven refresh — is covered in UI and displayers (`08-ui-and-displayers.pdf`).

Configure audio and editor helpers

Configure economy audio

`AudioData` identifies a clip, category, and volume. Definition assets expose operation-specific hooks for buying, unlocking, level-up, production, merge, claim, activation, and travel, and normal manager operations invoke the configured feedback automatically. Forced or project-defined mutations can bypass standard audio; trigger project feedback explicitly on those paths. A null clip is valid and is skipped.

Add `AudioManager`, assign music/SFX sources, and use `AudioButton` for standard button feedback.

Persist mute settings

`EasyIdleGame.UI.AudioManager` owns the background-music and sound-effect mute state and raises `OnSettingsChanged`. Audio settings are not part of the economy `SaveFile`; persist mute preferences in your project's save or settings layer and subscribe/unsubscribe with the component lifecycle.

There is no dedicated audio demo. Inspect the `AudioData` hooks on definition assets in the Inspector, and use `DocsDemo` or `AdventureCommunist` for broader playback references.

Use the package drawers and validation warnings

The editor supplies custom inspectors or property drawers for `BigNumber`, `BuyAmount`, `Input`, `Output`, `DropTable`, `Lock`, upgrade blocks and filters, wrapper synchronization, comment areas, and demo readmes. Conditional messages identify likely authoring mistakes, but they are editor guidance rather than runtime enforcement — add runtime validation for content loaded or generated outside the Inspector.

Preserve serialized enum numeric values during migrations and use `FormerlySerializedAs` when renaming a serialized field.

Editor commands are under **Tools > EasyIdleGame**:

- **RedrawAll** redraws loaded `BaseDisplayer` components in the editor.
- **InspectorMode > All / NoDebug / DebugOnly** controls custom-inspector debug sections.
- **ExecuteTests** is deprecated and only logs a warning; use Unity Test Runner for the included Edit Mode and Play Mode assemblies.